

**trad4**

## Cuarta Entrega de la Práctica Final



Jorge Mejías Donoso 100495807 [100495807@alumnos.uc3m.es](mailto:100495807@alumnos.uc3m.es)

Alberto Menchén Montero 100495692 [100495692@alumnos.uc3m.es](mailto:100495692@alumnos.uc3m.es)

Grupo 82 - Ingeniería Informática

|                                                                                     |          |
|-------------------------------------------------------------------------------------|----------|
| <b>Memoria de Desarrollo Semanal – Práctica Final: Traductor Frontend y Backend</b> | <b>3</b> |
| 1. Introducción                                                                     | 3        |
| 2. Desarrollo del Traductor Frontend (trad4.y)                                      | 3        |
| 2.1 Evolución del trabajo                                                           | 3        |
| 2.2 Mejoras e Implementaciones específicas                                          | 3        |
| 2.3 Resultados                                                                      | 5        |
| 3. Desarrollo del Traductor Backend (back4.y)                                       | 5        |
| 3.1 Inicio del Backend                                                              | 5        |
| 3.2 Implementaciones realizadas                                                     | 5        |
| 3.3 Resultados                                                                      | 6        |
| 4. Problemas Afrontados                                                             | 6        |
| 5. Valoración Final                                                                 | 7        |

# Memoria de Desarrollo Semanal – Práctica Final: Traductor Frontend y Backend

## 1. Introducción

Durante las últimas semanas hemos trabajado en el desarrollo de un **traductor completo** desde un subconjunto de C hasta código ejecutable en notación postfix, pasando por un **frontend** que convierte C en **Lisp** y un **backend** que convierte Lisp en **Forth**.

La práctica ha sido dividida en dos partes:

- **Frontend:** Desarrollo y finalización del traductor `trad4.y`.
- **Backend:** Implementación inicial del traductor `back4.y`.

Cada etapa ha seguido rigurosamente las especificaciones indicadas en el enunciado.

## 2. Desarrollo del Traductor Frontend (`trad4.y`)

### 2.1 Evolución del trabajo

El archivo `trad4.y` parte del anterior `trad3.y`, que ya había abordado la traducción básica de programas en C hacia una representación intermedia en Lisp. En esta nueva fase se han añadido los siguientes bloques funcionales:

### 2.2 Mejoras e Implementaciones específicas

- **Variables Globales y Locales:**
  - Se ha mejorado la definición de variables globales como `(setq var 0)` o `(setq var valor)`.
  - Se ha añadido la concatenación de nombre de función y variable (`main_var`) para variables locales, gestionando adecuadamente los ámbitos.
- **Funciones:**

- Soporte para funciones sin parámetros, con un parámetro y con múltiples parámetros.
- Traducción a definiciones Lisp mediante `(defun nombre_funcion (parametros) cuerpo)`.
- **Sentencias Print y Printf:**
  - Implementación de `puts` y `printf` (con múltiples argumentos).
  - Traducción correcta a `(print ...)` y `(princ ...)`, respetando las diferencias de comportamiento entre ambos.
- **Estructuras de Control:**
  - Traducción de `while` a `(loop while expr do cuerpo)`.
  - Traducción de `if` simple y `if-else` a expresiones Lisp, incluyendo uso de `progn` para múltiples sentencias.
- **Operadores y Expresiones:**
  - Implementación de operadores aritméticos, lógicos y de comparación, respetando la precedencia y asociatividad de C.
- **Vectores:**
  - Traducción de vectores C (`int v[5];`) a Lisp `((setq v (make-array 5)))`.
  - Traducción de acceso y modificación mediante `aref` y `setf`.
- **Control de Retornos:**
  - Traducción de sentencias `return` en funciones, adecuando su posición en el código Lisp.
- **Integración con las instrucciones `//@`**

- Gestión de comentarios especiales `//@` para la ejecución automática de pruebas (`//@ (main)`).

## 2.3 Resultados

La implementación completa de `trad4.y` permite ejecutar programas C de prueba traducidos correctamente a Lisp, cumpliendo todas las fases requeridas en la especificación, y pudiendo ser evaluados mediante `clisp`.

## 3. Desarrollo del Traductor Backend (`back4.y`)

### 3.1 Inicio del Backend

Una vez finalizado el frontend, se ha comenzado a desarrollar el **backend** en el archivo `back4.y`, encargado de traducir el código Lisp generado a código Postfix (Forth).

### 3.2 Implementaciones realizadas

- **Reconocimiento de Constructos Básicos:**
  - Traducción directa de definiciones (`defun nombre () cuerpo`) a : `nombre ... ;`.
  - Traducción de `main` como función especial : `main ... ;`.
- **Variables:**
  - Declaración de variables globales usando `variable var`.
  - Inicialización de variables mediante `<valor> <var> !`.
- **Operaciones Básicas:**
  - Traducción de operaciones aritméticas, lógicas y de comparación respetando la sintaxis postfix (ej: `2 3 *` para `(* 2 3)`).

- **Estructuras de Control:**

- Traducción de `if` y `if-else` usando `if ... else ... then` en Forth.
- Traducción de bucles `while` usando `begin ... while ... repeat`.

- **Impresión:**

- Traducción de `(print ...)` a `." string"` y de `(princ expr)` a `expr .` en Forth.

### 3.3 Resultados

El backend desarrollado permite tomar el código Lisp generado por el frontend y transformarlo en instrucciones ejecutables en Forth. Las pruebas realizadas han mostrado que la ejecución en `gforth` ofrece resultados coherentes con la ejecución original del código C.

## 4. Problemas Afrontados

Durante el desarrollo del archivo `back4.y` hemos tenido que afrontar algunos **warnings** generados por Bison, principalmente relacionados con **conflictos menores** de reducción o ambigüedades en la gramática al procesar expresiones Lisp complejas.

A lo largo del trabajo, hemos realizado varias revisiones y refactorizaciones para **reducir el número de advertencias**, consiguiendo corregir una gran parte de ellas.

Sin embargo, aún persisten algunos **warnings**, que no afectan directamente al funcionamiento básico del backend, pero que marcaremos como **objetivo prioritario de mejora** para la siguiente entrega del proyecto.

Nuestro propósito es eliminar completamente estos conflictos en las próximas versiones para garantizar una implementación más robusta y limpia.

## 5. Valoración Final

Este proyecto ha permitido:

- Consolidar los conocimientos en diseño de gramáticas y traducción de lenguajes utilizando **Bison**.
- Entender el funcionamiento de **lenguajes basados en pila** como Forth.
- Afrontar la construcción de traductores **por etapas**, primero hacia un lenguaje intermedio (Lisp) y luego hacia código máquina (Postfix).
- Gestionar problemas de ambigüedad, control de ámbitos y transformación de estructuras complejas como vectores y funciones.

Se ha trabajado de forma progresiva, cumpliendo todas las especificaciones del enunciado y solucionando los problemas conforme han ido apareciendo, desarrollando tanto el frontend como el backend de manera coordinada.